

**DEPARTMENT OF COMPUTER SCIENCE & INFORMATION
TECHNOLOGY**

Bachelor of Science in Computer Science & IT



Design & Implementation Report

Course: Compiler Design (CT-465)

Team:

Areeba Batool (CT-22059)

Syeda Shinza Wasif (CT-22063)

Asifa Siraj (CT-22070)

Submitted to: Miss Firdous Riaz

Submission Date: 28th April, 2026

Table of Contents

Table of Contents.....	1
1. Introduction & Problem Statement.....	2
1.1 Problem Statement.....	2
1.2 Learning Outcomes Addressed.....	2
2. Assumptions & Language Scope.....	3
2.1 Supported Language Features.....	3
2.2 Compiler Architecture Assumptions.....	4
3. Formal Design.....	4
3.1 Finite State Machine (FSM) — Lexical Analysis.....	4
3.2 Context-Free Grammar (CFG).....	6
3.3 Symbol Table Structure & Semantic Checks.....	10
4. Implementation.....	11
4.1 Overall Architecture.....	11
4.2 Phase 1: Lexer (Lexical Analysis).....	12
4.3 Phase 2: Parser (Syntax Analysis).....	12
4.4 Phase 3+4: Single-Pass Compiler (Analysis + IR Generation).....	13
4.5 Phase 5: Assembly Code Generator.....	15
4.6 Phase 6: Virtual Machine (IR Interpreter).....	16
5. Sample Programs & Compiler Output.....	17
6. Research & Theoretical Foundation.....	18
7. Testing & Validation.....	19
8. Conclusion & Future Work.....	21

1. Introduction & Problem Statement

Compiler design is a cornerstone of computer science, involving the systematic translation of high-level human-readable programming languages into machine-executable instructions. This report documents the complete design and implementation of the Lightweight Compiler, a full-pipeline compiler targeting a simplified C++-style programming language intended for embedded systems and educational purposes.

The compiler was built for the Compiler Design course (CT-465) as a complex computing problem that spans formal language theory, data structures, algorithm design, and systems programming. The implementation covers all six classical compiler phases: Lexical Analysis, Syntax Analysis, Semantic Analysis, IR Generation, Assembly Code Generation, and Virtual Machine Execution making it a complete end-to-end compiler pipeline.

1.1 Problem Statement

Design and implement a compiler for a simplified programming language incorporating appropriate functionalities to translate high-level code into an executable format.

Our project aims to develop a lightweight programming language for embedded systems. The language supports basic constructs such as integer and boolean variables, conditionals (if-else), loops (while, for), and user-defined functions. The compiler must:

- Generate Intermediate Representation (IR) or assembly code for a hypothetical virtual machine
- Provide user-friendly error messages for syntax and semantic errors
- Ensure efficient execution on resource-constrained devices
- Handle a well-defined subset of language features with the flexibility to extend functionality

Link of project: <https://github.com/AsifaSiraj/Compiler-Design-CPP.git>

1.2 Learning Outcomes Addressed

CLO/CPA	Attribute	How Addressed in This Project
CLO 1	Design & Implement	Full 6-phase pipeline from source text to virtual machine execution
CPA-2	Depth of Analysis	CFG, FSM, symbol table, IR each requiring abstract modeling and innovative analysis
CPA-3	Depth of Knowledge	Recursive-descent parsing, scoped symbol tables, three-address code IR, x86-64 assembly generation

CPA-9	Requirement ID	Language features, error categories, and target architecture were identified from ill-defined real-world constraints
-------	----------------	--

2. Assumptions & Language Scope

The following design decisions and assumptions were made to bound the problem scope while producing a feature-rich and practically runnable compiler.

2.1 Supported Language Features

Feature	Status	Details
Data Types: int, bool	Supported	Integer (32-bit) and boolean (true/false) variables
String Literals	Supported	Read-only string constants for print output; escape sequences <code>\n</code> <code>\t</code>
Arithmetic Operators	Supported	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> with proper operator precedence
Comparison & Logic	Supported	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>&&</code> , <code> </code> , <code>!</code>
if / else	Supported	Full if-else with nested blocks
while loop	Supported	With break and continue support
for loop	Supported	C-style: init; condition; increment in same scope
Functions (void/int/bool)	Supported	With typed parameters, return values, and recursion
Variable Shadowing / Scoping	Supported	Block-scoped variables, inner scopes shadow outer ones
cout	Supported	Cout outputs to console / program output
Floating-point numbers	Not supported	Intentionally excluded for embedded system simplicity
Arrays / Pointers	Not supported	Beyond current scope; reserved for future extension
input() built-in	Lexed only	Token recognized; runtime execution not implemented

2.2 Compiler Architecture Assumptions

- Implementation Language: JavaScript (ES2020+), runnable in both Node.js and browser environments
- Single-pass combined semantic analysis and IR generation for reduced memory usage
- Target output: Three-Address Code (TAC) IR for a stack-based virtual machine
- Secondary output: x86-64 pseudo-assembly for educational reference
- Error handling: Syntax errors halt compilation; semantic errors are collected and reported together
- Execution model: In-memory virtual machine with a 500,000 step guard against infinite loops
- No compiler optimization passes single-pass architecture prioritizes simplicity and embedded suitability

3. Formal Design

3.1 Lexical Analysis

The lexer implements a deterministic finite automaton (DFA) that recognizes all token categories in the language.

3.1.1 Token Categories & Regular Expressions

Token Category	Regular Expression	Examples
INT_LITERAL	<code>[0-9]+</code>	0, 42, 1024, 65535
IDENTIFIER	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	x, myVar, counter_1
STRING_LITERAL	<code>`"([^"\\"])*`</code>	<code>\.)*</code>
KEYWORD	<code>int bool void string if else while for return break continue cout cin true false</code>	int, while, cout, cin

ARITHMETIC_OP	[+\-*/%]	+, -, *, /, %
COMPARE_OP	== != <= >= < >	==, !=, <=
LOGICAL_OP	&& !	&&, , !
ASSIGN	= (not ==)	x = 5
OUT_OP	<<	cout << x
IN_OP	>>	cin >> x

3.1.2 Key Implementation Details

The Lexer class maintains a position pointer (pos), current line, and column. The advance() method consumes one character and updates line/column tracking. The match() method peeks ahead for two-character operators. The skipWC() method handles both line comments (//) and block comments (/* */) before every token scan.

3.2 Context-Free Grammar (CFG)

The following CFG defines the complete syntax of the Compiler. The grammar is LL(1)-compatible and suitable for recursive descent parsing. Terminal symbols are shown in bold/monospace.

3.2.1 START SYMBOL

Program → DeclarationList

3.2.2 DECLARATIONS

DeclarationList → Declaration DeclarationList | ε

Declaration → VarDecl | FuncDecl | Statement

3.2.3 TYPES

Type → int | bool | void

3.2.4. VARIABLE DECLARATION

VarDecl → Type IDENTIFIER ; | Type IDENTIFIER = Expression ;

3.2.5. FUNCTION DECLARATION

FuncDecl $\rightarrow$ Type IDENTIFIER (ParamList) Block

ParamList $\rightarrow$ Param MoreParams | ϵ

MoreParams $\rightarrow$, Param MoreParams | ϵ

Param $\rightarrow$ Type IDENTIFIER

3.2.6. BLOCK

Block $\rightarrow$ { DeclarationList }

3.2.7. STATEMENTS

Statement $\rightarrow$ IfStmt | WhileStmt | ForStmt | ReturnStmt | BreakStmt
| ContinueStmt | PrintStmt | InputStmt | Block | ExprStmt

3.2.8. CONTROL STATEMENTS

- **IF**

IfStmt $\rightarrow$ if (Expression) Statement ElsePart

ElsePart $\rightarrow$ else Statement | ϵ

- **WHILE**

WhileStmt $\rightarrow$ while (Expression) Statement

- **FOR**

ForStmt $\rightarrow$ for (ForInit ; ForCond ; ForIncr) Statement

ForInit $\rightarrow$ VarDecl | ExprStmt | ϵ

ForCond $\rightarrow$ Expression | ϵ

ForIncr $\rightarrow$ Expression | ϵ

3.3.9. OTHER STATEMENTS

ReturnStmt $\rightarrow$ return Expression ; | return ;

BreakStmt $\rightarrow$ break ;

ContinueStmt $\rightarrow$ continue ;

PrintStmt $\rightarrow$ cout << ExprList ;

InputStmt $\rightarrow$ cin >> VarList ;

ExprStmt $\rightarrow$ Expression ;

3.2.10. LISTS

ExprList $\rightarrow$ Expression ExprListTail

ExprListTail $\rightarrow$ << Expression ExprListTail | ϵ

VarList $\rightarrow$ IDENTIFIER VarListTail

VarListTail $\rightarrow$ >> IDENTIFIER VarListTail | ϵ

3.2.11. EXPRESSIONS (PRECEDENCE BASED CFG)

- **TOP LEVEL**

Expression $\rightarrow$ Assignment

- **ASSIGNMENT**

Assignment $\rightarrow$ IDENTIFIER = Assignment | LogicalOr

- **LOGICAL OR**

LogicalOr $\rightarrow$ LogicalAnd LogicalOrTail

LogicalOrTail $\rightarrow$ || LogicalAnd LogicalOrTail | ϵ

- **LOGICAL AND**

LogicalAnd $\rightarrow$ Equality LogicalAndTail

LogicalAndTail $\rightarrow$ && Equality LogicalAndTail | ϵ

- **EQUALITY**

Equality $\rightarrow$ Relational EqualityTail

EqualityTail $\rightarrow$ == Relational EqualityTail | != Relational EqualityTail | ϵ

- **RELATIONAL**

Relational $\rightarrow$ Additive RelationalTail

RelationalTail $\rightarrow$ < Additive RelationalTail | > Additive RelationalTail

| <= Additive RelationalTail | >= Additive RelationalTail | ϵ

- **ADDITION / SUBTRACTION**

Additive $\rightarrow$ Multiplicative AdditiveTail

AdditiveTail $\rightarrow$ + Multiplicative AdditiveTail | - Multiplicative AdditiveTail | ϵ

- **MULTIPLICATION / DIVISION**

Multiplicative $\rightarrow$ Unary MultiplicativeTail

MultiplicativeTail $\rightarrow$ * Unary MultiplicativeTail | / Unary MultiplicativeTail
| % Unary MultiplicativeTail | ϵ

- **UNARY**

Unary $\rightarrow$! Unary | - Unary | Primary

- **PRIMARY EXPRESSIONS**

Primary $\rightarrow$ INT_LITERAL | STRING_LITERAL | true | false | IDENTIFIER
| IDENTIFIER (ArgList) | (Expression)

- **FUNCTION CALL ARGUMENTS**

ArgList $\rightarrow$ Expression ArgListTail | ϵ

ArgListTail $\rightarrow$, Expression ArgListTail | ϵ

3.2.12. TERMINALS (TOKENS)

int, bool, void

if, else, while, for

return, break, continue

cout, cin

true, false

IDENTIFIER

INT_LITERAL

STRING_LITERAL

Operators:

+ - * / % = == != < > <= >= && || !

Delimiters:

() { } ; , < >

3.2.13 Operator Precedence Table

Level	Operators	Associativity	Production Rule
-------	-----------	---------------	-----------------

1 (lowest)	=	Right	assignExpr
2		Left	orExpr
3	&&	Left	andExpr
4	== !=	Left	equalExpr
5	< > <= >=	Left	relExpr
6	+ -	Left	addExpr
7	* / %	Left	mulExpr
8 (highest)	! - (unary)	Right	unaryExpr

3.3 Symbol Table Structure & Semantic Checks

The symbol table is implemented as a chain of scoped hash maps. Each scope level maintains its own map from identifier names to symbol descriptors. Scope chains enable variable shadowing and lexical scoping identical to C/C++.

3.3.1 SymbolTable Class Design

```
class SymbolTable {
  parent: SymbolTable | null    // Enclosing scope (null = global)
  table: Object<string, SymbolInfo> // Plain hash object (not Map)

  define(name, info) // Add to current scope
  lookup(name)       // Search current + all parent scopes recursively
  has(name)          // Check current scope only (for redeclaration detection)
}

interface SymbolInfo {
  varType: 'int' | 'bool' | 'void' | 'string' // ← add 'string'
  kind: 'var' | 'func' | 'param'
  params?: { name: string, type: string }[] // only for kind='func'
}
```

3.3.2 Scope Management

Scopes are pushed/popped at the following boundaries:

Construct	Push Scope At	Pop Scope At
Block { }	Opening {	Closing }
Function declaration	After registering function name in parent scope	After processing body

for loop	Before init clause (own scope for loop var)	After loop body
----------	---	-----------------

3.3.3 Semantic Checks Performed

Check	Error Emitted
Undeclared variable used	Undeclared variable 'x' at line N
Redeclaration in same scope	Variable 'x' already declared in this scope
Type mismatch in assignment	Cannot assign 'bool' to 'int'
Type mismatch in arithmetic	Arithmetic op '+' requires int operands
Comparison type mismatch	Comparison '<' requires int operands
Equality on mismatched types	Equality '==' on mismatched types 'int' and 'bool'
Undeclared function call	Undeclared function 'foo'
Calling a non-function	'x' is not a function
Wrong argument count	Function 'foo' expects 2 args, got 3
Argument type mismatch	Arg 1 type mismatch in call to 'foo'
Return type mismatch	Return type mismatch: expected 'int', got 'bool'
Return outside function	Return outside function
break outside loop	'break' outside loop
continue outside loop	'continue' outside loop
Unary negation on non-int	Unary '-' requires int
cin used on undeclared variable	Undeclared variable 'x'

4. Implementation

The compiler is implemented in JavaScript as a single self-contained module that can execute in both Node.js and web browser environments. It consists of six distinct phases organized as separate classes, all orchestrated by a top-level compile() function.

4.1 Overall Architecture

#	Phase	Class(es)	Description
---	-------	-----------	-------------

1	Lexical Analysis	Lexer	Converts source text into a flat token stream; handles keywords, operators, literals, comments
2	Syntax Analysis	Parser	Recursive descent parser; builds a typed Abstract Syntax Tree (AST) from the token stream
3+4	Semantic + IR Gen	SinglePassCompiler	Single-pass traversal performs type checking, scope analysis, and generates Three-Address Code IR simultaneously
5	Assembly Gen	AsmGen	Converts IR instructions to x86-64 pseudo-assembly; for reference and educational output
6	VM Execution	VM	Interprets the IR instruction set; handles arithmetic, control flow, function calls, and I/O

4.2 Phase 1: Lexer (Lexical Analysis)

The Lexer class accepts a source string and produces an array of Token objects. Each token carries its type, value, line number, and column — enabling precise error reporting.

4.2.1 Token Class

```
class Token {
  type: string // TokenType constant (e.g., 'int', 'IDENTIFIER')
  value: any   // Actual value (number for INT_LITERAL, boolean for true/false)
  line: number // 1-based line number
  col: number  // 1-based column number
}
```

4.2.2 Lexer Algorithm

The main tokenize() loop follows this flow on each iteration:

1. Call skipWC() to consume whitespace and both comment forms
2. Check for end-of-file; emit EOF token and exit
3. Peek at current character; dispatch to specialized reader:
 - Digit [0-9] → readNum() — collects consecutive digits, parses as integer
 - Alpha [a-zA-Z_] → readId() — collects identifier, checks KEYWORDS map for keyword type
 - Quote " → readStr() — collects string body with escape sequence processing
 - Any other → single-character or two-character operator dispatch in switch statement

4. Push the produced Token into the tokens array; continue loop

4.2.3 Error Handling

The lexer throws a `LexerError` (extends `Error`) immediately on encountering an unknown character or unterminated string literal. The error includes phase name ('Lexer'), line, and column for precise diagnostics.

4.3 Phase 2: Parser (Syntax Analysis)

The Parser class implements a hand-written recursive descent parser. It accepts the token array from the Lexer and produces a fully typed AST using a hierarchy of 20+ AST node classes.

4.3.1 AST Node Hierarchy

AST Node Class	Extends	Key Fields
<code>ProgramNode</code>	<code>ASTNode</code>	<code>declarations: ASTNode[]</code>
<code>FuncDeclNode</code>	<code>ASTNode</code>	<code>retType, name, params[], body: BlockNode</code>
<code>VarDeclNode</code>	<code>ASTNode</code>	<code>varType, name, init: ASTNode null</code>
<code>BlockNode</code>	<code>ASTNode</code>	<code>statements: ASTNode[]</code>
<code>IfNode</code>	<code>ASTNode</code>	<code>condition, thenBranch, elseBranch</code>
<code>WhileNode</code>	<code>ASTNode</code>	<code>condition, body</code>
<code>ForNode</code>	<code>ASTNode</code>	<code>init, condition, increment, body</code>
<code>ReturnNode</code>	<code>ASTNode</code>	<code>value: ASTNode null</code>
<code>BreakNode</code>	<code>ASTNode</code>	—
<code>ContinueNode</code>	<code>ASTNode</code>	—
<code>PrintNode</code>	<code>ASTNode</code>	<code>args: ASTNode[]</code>
<code>InputNode</code>	<code>ASTNode</code>	<code>ars: VarNode[]</code>
<code>ExprStmtNode</code>	<code>ASTNode</code>	<code>expression: ASTNode</code>
<code>BinOpNode</code>	<code>ASTNode</code>	<code>op: string, left, right</code>
<code>UnaryNode</code>	<code>ASTNode</code>	<code>op: string, expr</code>
<code>AssignNode</code>	<code>ASTNode</code>	<code>name: string, value: ASTNode</code>
<code>CallNode</code>	<code>ASTNode</code>	<code>name: string, args: ASTNode[]</code>
<code>VarNode</code>	<code>ASTNode</code>	<code>name: string</code>

AST Node Class	Extends	Key Fields
IntLitNode / BoolLitNode / StringLitNode	ASTNode	value: number boolean string

4.3.2 Parser Methods

The parser uses the following key methods:

- peek() / advance() / prev(): Token stream navigation
- match(...types): Conditional consume if next token matches any given type
- consume(type, msg): Mandatory consume; throws ParseError with friendly message on mismatch
- parseDecl() → parseTyped() / parseStmt(): Top-level dispatch
- parseExpr() → parseAssign() → parseOr() → ... → parsePrimary(): Expression precedence chain
- ParseError: Thrown immediately on syntax violation; includes line number and unexpected token

4.4 Phase 3+4: Single-Pass Compiler (Semantic Analysis + IR Generation)

The SinglePassCompiler extends IRGen and combines semantic analysis with IR emission into one AST traversal. This is a deliberate architectural choice for embedded system compilers where two passes would double memory usage.

4.4.1 Single-Pass Design Rationale

Single-pass compilation reduces memory usage at the cost of certain optimizations. Since the language has no forward declarations of variables (only functions declared before use), the single-pass model is sound and correct for this language design.

4.4.2 IR Instruction Set

The Three-Address Code (TAC) instruction set consists of the following operation codes:

Opcode	arg1	arg2	result / purpose
DECLARE	type (int bool string)	—	result = variable name; allocate slot
ASSIGN	source (place/literal)	—	result = destination; copy value
ADD/SUB/MUL/DIV/MOD	left operand	right operand	result = temp variable t1, t2, ...
EQ/NEQ/LT/GT/LTE/GTE	left operand	right operand	result = temp; produces 0 or 1
AND/OR	left operand	right operand	result = temp; logical 0 or 1
NEG/NOT	operand	—	result = temp; unary negate/complement
LABEL	label name	—	emit a jump target label
GOTO	—	—	result = label; unconditional jump
IF_FALSE	condition var	—	result = label; jump if condition == 0
FUNC_BEGIN / FUNC_END	function name	—	function boundaries in IR
PARAM	type	—	result = param name; declare param
ARG	argument value	—	push argument for upcoming CALL
CALL	function name	arg count	result = temp; return value destination
RETURN	return value / null	—	exit current function
PRINT	args joined by	—	program console via cout <<
BREAK / CONTINUE	—	—	loop control; resolved by VM
INPUT	—	—	result = variable name; reads value from cin >>, prompts user at runtime
BOOL_LITERAL	—	—	result = 0 or 1; emitted for true/false literals inline

4.4.3 IR Generation Examples

The following examples show source code and corresponding TAC IR output:

```
// Source:
int x = 5;
int y = x + 3;

// Generated IR:
000: x = 5
001: t1 = x ADD 3
002: y = t1

// Source:
if (x > 0) { print(x); } else { print(0); }

// Generated IR:
000: t1 = x GT 0
001: IF_FALSE t1 GOTO else1
002: PRINT x
003: GOTO endif2
004: else1:
005: PRINT 0
006: endif2:

// Source:
while (i < 10) { i = i + 1; }

// Generated IR:
000: while1:
001: t1 = i LT 10
002: IF_FALSE t1 GOTO endwhile2
003: t2 = i ADD 1
004: i = t2
005: GOTO while1
006: endwhile2:
```

4.5 Phase 5: Assembly Code Generator

The AsmGen class translates TAC IR instructions into x86-64 pseudo-assembly. While this output is not directly executable (it lacks a full ABI and OS linkage), it demonstrates the mapping from IR to real hardware instructions and serves as an educational artifact.

4.5.1 Key Assembly Patterns

IR Op	Generated Assembly	Notes
ASSIGN	mov rax, <src> mov [rbp-N], rax	Stack slot N computed per variable

ADD	<code>mov rax, <a> mov rbx, add rax, rbx mov [rbp-N], rax</code>	All arithmetic via rax/rbx registers
DIV	<code>mov rax, <a> cqo mov rbx, idiv rbx mov [rbp-N], rax</code>	cqo sign-extends rax into rdx:rax
EQ (==)	<code>mov rax, <a> cmp rax, sete al movzx rax, al mov [rbp-N], rax</code>	sete sets AL to 1/0 after comparison
IF_FALSE	<code>mov rax, <cond> test rax, rax jz <label></code>	jz = jump if zero (false)
FUNC_BEGIN	<code><name>: push rbp mov rbp, rsp sub rsp, 256</code>	Standard x86 function prologue
CALL	<code>call <name> mov [rbp-N], rax</code>	Return value arrives in rax

4.6 Phase 6: Virtual Machine (IR Interpreter)

The VM class interprets the TAC instruction list. It maintains a label/function map built at startup for O(1) jump resolution, an environment (hash map) for variable storage, and a call stack for function invocations.

4.6.1 VM Execution Model

- Label Map: Built at construction by scanning all LABEL and FUNC_BEGIN instructions
- Environment: A plain JavaScript object maps variable names to their current values
- Function calls: `execFunc()` creates a new local environment, binds parameters, and executes the function body inline (not on a hardware stack)
- Resolve: The `res()` helper converts a 'place' string — either a numeric literal, a quoted string, or a variable name — to its runtime value
- Step limit: 500,000 instructions maximum before throwing an infinite loop error
- top-level execution: Instructions outside functions are executed directly; FUNC_BEGIN blocks are skipped until explicitly called

4.6.2 VM Execution Example: Fibonacci

Running the Fibonacci example generates a TAC IR of approximately 30 instructions and executes in under 2ms on a modern browser, printing the first 10 Fibonacci numbers correctly.

5. Sample Programs & Compiler Output

5.1 Hello World

```
int main() {  
    int x = 42;  
    int y = 8;  
    int sum = x + y;  
    print("Hello, ABC Tech World!");  
    print("The answer is:", sum);  
    bool flag = true;  
    if (flag) { print("Flag is true!"); }  
    return 0;  
}
```

Output:

```
Hello, ABC Tech World!  
The answer is: 50  
Flag is true!
```

5.2 Fibonacci Sequence

```
int main() {  
    int n = 10;  
    int a = 0; int b = 1; int i = 0; int temp = 0;  
    while (i < n) {  
        print(a);  
        temp = a + b; a = b; b = temp;  
        i = i + 1;  
    }  
    return 0;  
}
```

Output: 0 1 1 2 3 5 8 13 21 34

5.3 Recursive Factorial

```
int factorial(int n) {  
    if (n <= 1) { return 1; }  
    return n * factorial(n - 1);  
}  
int main() {  
    int i = 1;  
    while (i <= 8) {  
        print(i, "! =", factorial(i));  
        i = i + 1;  
    }  
    return 0;  
}
```

Output: 1! = 1 2! = 2 3! = 6 4! = 24 5! = 120 ... 8! = 40320

5.4 Variable Scoping Demo

```
int main() {  
    int x = 10;  
    { int x = 20; { int x = 30; print("Innermost:", x); } print("Inner:", x); }  
    print("Outer:", x);  
    return 0;  
}
```

Output: Innermost: 30 Inner: 20 Outer: 10

This verifies that the scoped symbol table correctly implements lexical scoping with shadowing.

5.5 Semantic Error Detection

```
// Uncommenting any of these lines produces a specific error:  
// int x = true;      → Cannot assign 'bool' to 'int'  
// print(undefined_var); → Undeclared variable 'undefined_var'  
// int r = add(1, 2, 3); → 'add' expects 2 args, got 3  
// break;             → 'break' outside loop
```

6. Research & Theoretical Foundation

6.1 Formal Language Theory

The compiler design is grounded in Chomsky's formal language hierarchy. The lexical layer uses regular languages (Type 3) recognized by a DFA. The syntactic layer uses context-free languages (Type 2) recognized by a pushdown automaton in practice, implemented as a recursive descent parser which is equivalent to a top-down PDA.

6.2 Parsing Strategy: Recursive Descent

Recursive descent parsing was chosen over table-driven alternatives (LL(1) parser tables, LALR(1) as used in yacc/bison) for several reasons:

- Transparency: Each grammar rule maps 1:1 to a parsing function, making the code self-documenting
- Error recovery: Manual control allows descriptive error messages with expected-vs-got context
- Flexibility: Handling complex constructs like if/else ambiguity is straightforward
- No external tools: The entire parser is hand-written, removing dependency on parser generators

The grammar was verified to be LL(1) for all non-expression productions. Expression parsing uses the precedence-climbing technique (iterative left-recursion elimination) to handle the full operator precedence hierarchy without left-recursion.

6.3 Intermediate Representation Design

Three-Address Code (TAC) was selected as the IR format. TAC is the industry-standard intermediate form used in production compilers (GCC, LLVM). Each TAC instruction has at most one operator, two source operands, and one destination — making it trivial to map to real machine instructions during code generation.

The TAC format used here is a linearized sequence of objects rather than a basic-block graph, which simplifies the VM interpreter at the cost of some optimization potential. This trade-off is appropriate for the single-pass, embedded-target design goals.

6.4 Symbol Table & Type System

The type system supports four primitive types: int, bool, void, and string (string is a read-only literal type). Type compatibility rules follow a simple subset of C's rules: int and bool are assignment-compatible (bool is widened to int). The scoped symbol table implements a standard linked-environment model identical to the approach described in the Dragon Book (Aho, Lam, Sethi, Ullman — Compilers: Principles, Techniques, and Tools).

6.5 References

5. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.
6. Nisan, N., Schocken, S. (2005). The Elements of Computing Systems. MIT Press. (Chapters on Lexical Analysis, Parsing, Code Generation)
7. Grune, D., van Reeuwijk, K., Bal, H.E., Jacobs, C.J.H., Langendoen, K. (2012). Modern Compiler Design (2nd ed.). Springer.
8. Pratt, V. (1973). Top Down Operator Precedence. ACM SIGPLAN Symposium on Principles of Programming Languages.
9. Mozilla Developer Network (MDN) — JavaScript Engine Internals.
<https://developer.mozilla.org/>
10. NCEAC Manual 2021 — Complex Problem Attributes Reference.
<https://www.seoulaccord.org/>

7. Testing & Validation

7.1 Test Categories

Test Category	Test Cases	Result
Lexer valid tokens	All 30+ token types, including escape sequences in strings, multi-char operators	PASS
Lexer errors	Unknown character @, unterminated string, lone & and	PASS (error reported)
Parser valid programs	All 7 built-in examples: Hello World, Fibonacci, Factorial, Bubble Sort, Primes, Scope, Error Demo	PASS
Parser error recovery	Missing semicolons, mismatched parentheses, invalid assignment target	PASS (clear messages)
Semantic type checks	All 15 semantic check categories listed in Section 3.3.3	PASS
Semantic scoping	Variable shadowing, break/continue outside loops, return outside function	PASS
IR generation	Correct TAC for if/else, while, for, function calls, recursion	PASS

VM execution arithmetic	All 5 operators; integer division truncation; modulo; division by zero error	PASS
VM execution recursion	factorial(8) = 40320; fibonacci(10) correct; Collatz sequence	PASS
Infinite loop guard	while(true){} → VM throws after 500,000 steps	PASS

7.2 Performance Metrics

All examples were compiled and executed in a browser environment (Chrome 124). Performance was measured using the Performance.now() API.

Program	Lines	Tokens	IR Instrs	Exec Time	Status
Hello World	18	~80	~25	< 1 ms	OK
Fibonacci (n=10)	14	~60	~18	< 1 ms	OK
Factorial (1–8, recursive)	25	~130	~45	< 2 ms	OK
Prime Numbers (1–50)	40	~200	~70	< 3 ms	OK
Scope & Functions	60	~350	~110	< 4 ms	OK

8. Conclusion & Future Work

8.1 Summary of Achievements

This project successfully delivers a complete, working six-phase compiler for the ABC Tech lightweight language. The key achievements are:

- A fully functional Lexer that tokenizes the complete language including keywords, operators, literals, and comment forms, with precise line/column error reporting
- A recursive-descent Parser that builds a rich, fully typed AST and produces user-friendly syntax error messages
- A single-pass Semantic Analyzer that performs 15 distinct type-checking and scoping validations simultaneously with IR generation
- A Three-Address Code IR generator that correctly handles all language constructs including function calls, recursion, nested scopes, and all loop forms

- An x86-64 Assembly Code Generator that produces educational reference assembly from the IR
- A complete in-memory Virtual Machine that correctly executes all language programs, including recursive functions, with a configurable step-limit safety guard
- An interactive web-based IDE that visualizes all six pipeline stages with real-time compilation, a token viewer, AST explorer, IR viewer, and assembly output panel

8.2 Limitations

- No arrays or pointer types limits practical programs to scalar variables
- No floating-point arithmetic integers only
- No module system or imports all code in one file
- No optimizer IR is un-optimized; no dead code elimination or constant folding
- Assembly output is pseudo-assembly not directly assemblable with NASM/GAS without ABI/linkage additions
- No input() runtime support the token is lexed but has no VM implementation

8.3 Future Work

Extension	Description
Arrays & Fixed Memory	Add fixed-size integer arrays with index operators; introduce a simple memory model
Constant Folding	Evaluate constant expressions at compile time (e.g., $2+3 \rightarrow 5$) to reduce IR size
Dead Code Elimination	Remove IR instructions in unreachable branches (after unconditional return/goto)
Register Allocation	Replace temporary variables in assembly with actual CPU registers using graph coloring
Floating-point support	Add float/double types and generate SSE2 instructions in the assembly phase
Struct types	Simple record types for grouping related integer fields — useful for embedded hardware registers
WASM Backend	Target WebAssembly binary format for direct browser execution without a custom VM